

指令系统

程序既可用高级语言编写,亦可用机器语言编写;但计算机只能识别和执行用机器语言编写的程序。高级语言不能直接操纵底层硬件

各种高级语言编写的应用程序,最终都要翻译成机器语言来执行

机器语言是由一系列的指令(语句)组成的。

用机器语言编写的程序执行效率最高,速度最快(无需编译,计算机可直接识别)

指令的格式就是机器语言的语法。

每条指令规定机器完成一定的(比较大的)功能(例如完成一次算术/逻辑运算,完成一次移位/调用)(底层硬件来支撑)

一台计算机的所有的指令集合称为该机的指令系统或指令集。它是程序工作者编制程序的基本依据,也是进行计算机逻辑设计的基本依据。

从计算机组成的层次结构来说,计算机的指令有微指令、机器指令和宏指令之分。

微指令是微程序级的命令,属于硬件;宏指令是由若干条机器指令组成的软件指令,属于软件。

机器指令介于微指令与宏指令之间,通常简称为指令,每一条指令可完成一个独立的算术运算或逻辑运算操作。本章讨论的指令都是机器指令。

个人计算机多用CISC结构,(超大型计算机多用RISC结构)。

对指令系统的要求:

- ① **完备性**: 用汇编语言编写各种程序时,指令系统直接提供的指令足够使用,而不必用软件来实现。完备性要求指令系统丰富、功能齐全、使用方便。
- ② **有效性**: 利用该指令系统所编写的程序能高效率地运行。高效率主要表现在程序占据的存储空间小、执行速度快。一般地,一个功能更强、更完善的指令系统,必定有更好的有效性。

③ **规整性**: 包括指令系统的对称性、匀齐性、指令格式和数据格式的一致性

④ **兼容性**: 系列机各机种之间具有存储单元可 所有寄存器、一种操作 指令长度和数据长度有一定关系,
相同的基本结构和共同的基本指令集,同等对待。 性质的指 方便存取
因而指令系统是兼容的(向上兼容), 令可支持 与存储器长度呈倍数关系
各机种上基本软件可以通用。 多种数据
使用各种寻址方式 类型

低级语言与硬件的关系:

计算机语言有高级语言和低级语言之分

高级语言如 C, FORTRAN 等, 其语句和用法与具体机器的指令系统无关.

(通过编译系统^{很强大}将用高级语言写的程序变成机器语言目标程序)

低级语言分机器语言(二进制语言)和汇编语言(符号语言), 这两种语言都是面向机器的语言, 它们和具体机器的指令系统密切相关. ^{移植性很差, 不同的CPU不一样.}

^{mips 的不能移植到 arm 上.}

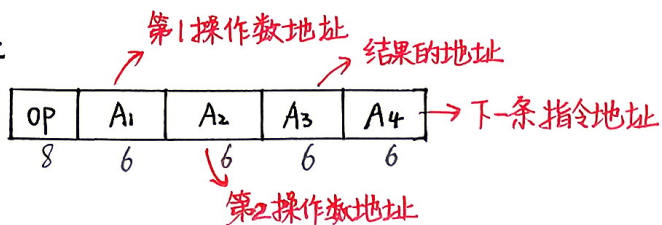
→ 如何操纵硬件? 高级语言中有与汇编语言之间的调用接口

利用堆栈来传递参数(地址), 把高级语言的源程序通过汇编/编译生成目标程序, 通过链接程序把高级语言源程序及其调用的底层的操纵硬件的程序链接成一个可执行文件.

汇编语言写的程序在操作系统里. 操作系统里集成了很多 API 函数(给高级语言来用)

地址码

① 四地址



$(A_1) OP(A_2) \rightarrow A_3$
4次访存

设计地址码时要考虑指令的执行时间以及访问内存的范围

设指令字长为32位, 操作码固定为8位, 剩下24位每个分6位.

操作数/运算结果只能写在0~63这最小的64个内存地址, 超出这个区间, 地址就无法表达, 找不到这个指令, 不可取.

寻址范围 $2^6 = 64$

怎样在不改变地址长度的前提下, 让地址码变少, 让地址码长度变长, 让寻址范围变大, 同时也尽量减少访问内存的次数, 让速度变快?

若增加一个硬件PC, 用它来代替下一条指令地址A₄

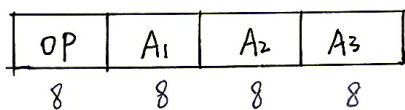
(用PC记录当前指令地址, 当前指令取出后把PC加1 (假如一条指令长度占1个编址单位))

75%的指令都是顺序执行, 25%的指令需要用到转移指令

当前指令地址的下一个存储单元被执行的概率是75%.

这样地址码可以少一个

② 三地址

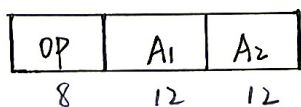


$(A_1) OP(A_2) \rightarrow A_3$
4次访存

寻址范围 $2^8 = 256$

若A₃用A₁或A₂代替

③ 二地址



$(A_1) OP(A_2) \rightarrow A_1$ 或 $(A_1) OP(A_2) \rightarrow A_2$
4次访存

寻址范围 $2^{12} = 4K$

专用寄存器, 可用操作码标记它, 不用在指令地址中给出

若结果存于ACC, 则3次访存, 速度更快. 若用ACC代替A₁(或A₂)

④ 一地址



无需访存
 $(ACC) OP(A_1) \rightarrow ACC$
2次访存

寻址范围 $2^{24} = 16M$

⑤ 零地址

无地址码，由操作码来指定专用寄存器中的操作数，还有一些指令确实不需要可表示一个或两个操作数的运算。
地址码，比如停机/空操作/等待指令。

若是双目运算，操作数可以在栈顶/次栈顶
若是单目运算，可在某个专用寄存器

若机器指令系统长度可变，那么地址码越少，指令越短，程序所占存储空间越小

若指令长度不变，则可扩大寻址范围，减少访存次数。

要加硬件加以辅助！

小结

当用一些硬件资源代替指令字中的地址码字段后，能够。

扩大指令的寻址范围、缩短指令字长、减少访存次数。

当指令的地址字段为寄存器时。

三地址 OP R_1, R_2, R_3

二地址 OP R_1, R_2

一地址 OP R_1

所有地址码都变成寄存器编号

寄存器编号若出现在指令中，说明它是通用寄存器

而专用寄存器一般一种类型只有一个，在指令中不用写，通过操作码来约定

并不代表只有一个。

另一个操作数可能是隐含的。

内存地址中5位只能寻址32个空间
离MB/GB差太远

用寄存器代替内存地址后，可以缩短指令长度。寄存器编号很短（32个，只要5位地址）。

这些指令执行阶段不访问内存，寄存器速度和运算器速度可认为是一样的。

指令助记符

ADD	加法
SUB	减法
MOV	传送
JMP	跳转
JSR	转子
STR	存储
LDA	读数

机器字长

机器字长简称字长,指计算机能直接处理的二进制数据的位数

字长是计算机中的一项重要技术指标,字长越长,计算和运算精度越高

字长可反映指令的直接寻址能力,若字长 n 位全用来寻址,可直接寻址 2^n 个字节

为便于处理字符数据及尽可能利用存储空间,一般把机器字长定为字节长度(8位)的整数倍(8/16/32/64)

微/小型机的字长多为8位、16位、32位,中/大型机的字长多为32、64位.因此,一个字中可以存储1个、2个、4个、8个字符.

指令字长

指令字的长度取决于操作码的长度、操作数地址的长度、操作数地址的个数

由于操作码的长度、操作数地址长度以及操作数地址数目不同,各种指令长度不固定.当然也不是任意的:

为充分利用存储空间,指令长度通常是存储器字长的倍数,指令字的长度也定为字节长度的整数倍

机器字长与指令字长的关系

机器字长度与指令字长度无固定关系,可以 $>|<|=$

- 单字长指令: 指令字长度 = 机器字长度
- 双字长指令: 指令字长度 = 2个机器字长度
- 半字长指令: 指令字长度 = 半个机器字长度

等字长指令结构^字 (结构简单,控制方便)

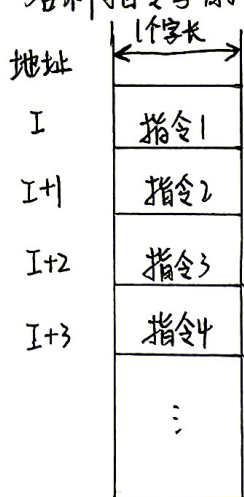
在同一个指令系统中,所有指令字长度都相等.

变字长指令结构 (结构灵活,能充分利用指令字长度,但指令控制较复杂).

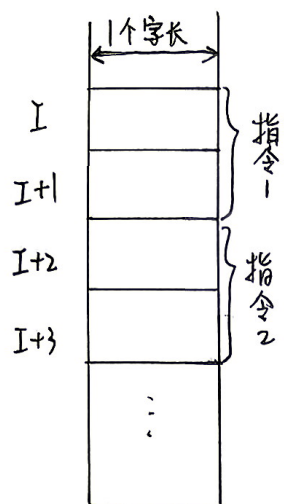
各种指令字长度随指令的功能而变化,有的指令是单字长指令,有的是双/三字长指令.

前提：一个存储单元长度 = 一个机器字长

各种指令字的结构情况



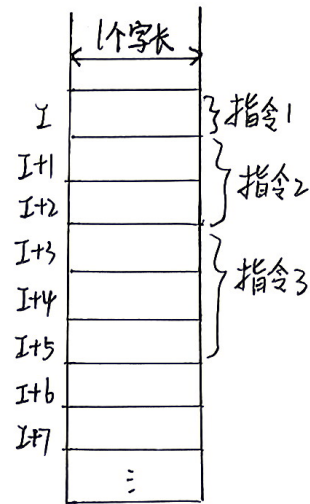
单字长指令



双字长指令

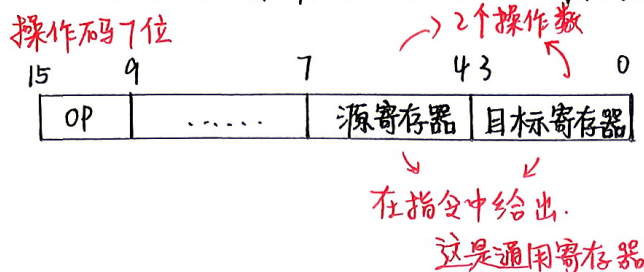


半字长指令



变字长指令

例. 假设机器字长是16位, 其中OP是操作码, 试分析指令格式的特点.



解:

① 单字长二地址指令

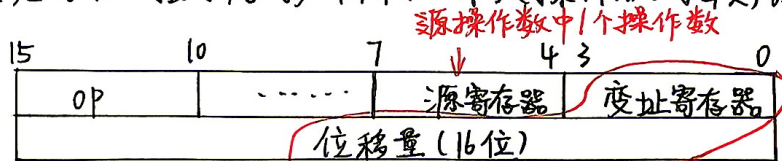
② 操作码字段OP可指定128条指令

③ 源寄存器和目标寄存器都是通用寄存器(可分别指定16个), 是RR型指令, 两个操作数均在寄存器中, 结果也送回寄存器

④ 执行过程中不访问内存, 速度特别快, 这种指令结构常用于算术逻辑运算类指令.

(RS型: 1个操作数在寄存器, 另1个在内存)
SS型: 2个操作数都在内存, 结果也写回内存

例. 假设机器字长是16位, 指令格式如下所示, OP是操作码字段, 试分析指令格式的特点.



2行: 32位长度

变址寻址. 操作数都在存储器中

解:

① 双字长二地址指令, 用于访问存储器(RS型指令).

② 操作码字段OP为6位, 可指定64种操作.

③ 一个操作数在源寄存器(有 $2^4=16$ 个通用寄存器来指定该操作数).

另一个操作数在存储器中(由变址寄存器和位移量决定), 是RS型指令.

通常用来访问内存.
取数/存数

机器指令格式

一条指令应包含如下信息:

- ① 进行何种操作: 即操作性质, 体现在指令中被称为 操作码
- ② 操作的对象: 数据来源以及如何寻找操作数, 体现在指令中被称为 地址码.
- ③ 操作结果: 结果存放在何处.

指令的一般格式

<u>指明操作性质</u> 操作码字段	<u>指明操作对象、对象寻找方式、操作结果放哪、下一条指令怎么找</u> 地址码字段
------------------------	---

1. 操作码

反映机器做什么操作

操作码所占的二进制位数决定了一台计算机所能允许的指令条数

- ① 固定长度: 操作码所占位数不变, 100条指令, 至少要7位^{最多} (何以有128条指令).
二进制

并且集中放在指令字的一个字段中.

这种格式有利于简化硬件设计, 减少指令译码时间, 广泛用于字长较长的大、中型计算机和超小型计算机中.

- ② 可变长度: 是操作码扩展技术的应用, 即操作码长度可变, 且分散地放在指令的不同字段中. 这种格式有利于压缩程序中操作码的平均长度, 在字长较短的微型机中被广泛应用.

如何让计算机能够区分操作码和地址码?

原则: 短操作码不可出现在长操作码中

即 短操作码不可作长操作码前缀. ★

设计短操作码时, 不要把所有二进制位用完, 至少要留1个作后面长操作码的前缀, 让译码系统好区分.

下图是一种扩展操作码的安排示意:

OP	A ₁	A ₂	A ₃
4位	4位	4位	4位

该计算机的指令码是16位字长 (即指令长度16位), 包括4位基本操作码字段和3个4位长的地址字段. 4位基本操作码全用于三地址指令, 也只有16条. 远远不够

因此, 我们必须向地址码字段扩展操作码长度.

扩展方法及步骤?

步骤① 15条三地址指令的操作码由4位基本操作码0000~1110所给定, 剩下一个1111则用于把操作码扩展到X地址码字段, 即由4位扩展到8位.

0000	XXXX	YYYY	ZZZZ
0001	XXXX	YYYY	ZZZZ
⋮	⋮	⋮	⋮
1110	XXXX	YYYY	ZZZZ

15条三地址指令

预留出一个组合"1111"作为窗口(为了能够区分), 作长操作码前缀

这里前面4位只能是短←
操作码没有用过的.

1111	0000	YYYY	ZZZZ
1111	0001	YYYY	ZZZZ
⋮	⋮	⋮	⋮
1111	1101	YYYY	ZZZZ

14条二地址指令

剩2个8位长度操作码11111110、11111111没有用, 就可以用来作更长长度操作码的前缀

步骤② 14条二地址指令的操作码由8位操作码的1111, 0000~1111, 1101 给定, 剩下的

1111, 1110 和 1111, 1111 又可用于把操作码扩充到Y地址字段, 即从8位又扩充到12位.

1111	1110	0000	ZZZZ
1111	1110	0001	ZZZZ
1111	1110	⋮	⋮
1111	1111	0000	ZZZZ
1111	1111	1110	ZZZZ

31条一地址指令

留出一个窗口, 给后面进一步扩展

步骤③ 31条一地址指令的操作码由12位操作码的1111, 1110, 0000~1111, 1111, 1110 给定,

剩下的 1111, 1111, 1111 扩充到Z地址码字段而形成16位操作码, 这时还可由

1111, 1111, 1111, 0000~1111, 1111, 1111, 1111 给出16条零地址指令

1111	1111	1111	0000
1111	1111	1111	0001
⋮	⋮	⋮	⋮
1111	1111	1111	1111

16条零地址指令

没有操作数了

76条指令

另一种扩展操作码技术(没有上一种多):

操作码的位数随地址数的减少而增加

	OP	A ₁	A ₂	A ₃	
4位操作码	0000	A ₁	A ₂	A ₃	最多15条三地址指令
	0001	A ₁	A ₂	A ₃	
	⋮	⋮	⋮	⋮	
	1110	A ₁	A ₂	A ₃	
8位操作码	1111 0000	A ₂	A ₃		最多15条二地址指令
	1111 0001	A ₂	A ₃		
	⋮	⋮	⋮	⋮	
	1111 1110	A ₂	A ₃		
12位操作码	1111 1111 0000	A ₃			最多15条一地址指令
	1111 1111 0001	A ₃			
	⋮	⋮	⋮	⋮	
	1111 1111 1110	A ₃			
16位操作码	1111 1111 1111 0000				16条零地址指令
	1111 1111 1111 0001				
	⋮	⋮	⋮	⋮	
	1111 1111 1111 1111				

操作码有的短,有的长,应当如何分配?

我们构造这种可变长度的操作码,就是想在不增加指令长度的前提下,
让指令功能更加丰富,让指令写出来的程序中操作码所占空间更小,让指令执行速度更快。
因此,产生了分配原则:

把短操作码分配给使用频率高的指令。

上述方法是在不增加指令长度的情况下,采用对地址少的指令使用较长的操作码,
对地址数多的指令使用较短的操作码。

实质: 增加指令数量,丰富指令功能。

在可变长操作码的指令系统设计中,究竟使用哪种扩展方法更好,指令的使用频度
(即在程序中出现的概率)是非常重要的依据。即:频度高的指令应分配短操作码,
频度低的指令应分配长操作码。既可有效缩短操作码在程序中的平均长度,节省存储空间,
又可缩短常用指令的译码时间以提高程序的运行速度。

缺点: 译码系统比固定操作码复杂,增加了设计控制器的难度,需要更多硬件支持。

操作数类型

地址 无符号整数

数字 定点数、浮点数、十进制数

字符 ASCII

逻辑数 逻辑运算

数据在存储器中的存放方式

地址按字节编,但访问时按字访问,存储器给的是字地址。
字=字节,若干个字节构成一个字

假设字长32位:

(4个字节构成)

1个字,则

字地址都是

4的倍数,

称“0号字”“4号字”...

小端方案:字地址为低字节地址

→认为右边最小,从右边取

大端方案:字地址为高字节地址

→认为左边最小,从左边取

例如:十六进制数01020304H从存储单元100H开始存储

十进制数6从存储单元104H开始存储

小端方案:字地址为低字节地址

大端方案:字地址为高字节地址

字地址	103H	102H	101H	100H
100H	01H	02H	03H	04H
104H	00H	00H	00H	06H
	107H	106H	105H	104H

字地址	100H	101H	102H	103H
100H	01H	02H	03H	04H
104H	00H	00H	00H	06H
	104H	105H	106H	107H

存放整数(32位长度)

例如:将字符串“ABCD”从存储单元100H开始存储

将十进制数1234的8421BCD码数从存储单元104H开始存储

小端方案:字地址为低字节地址

大端方案:字地址为高字节地址

字地址	103H	102H	101H	100H
100H	68H	67H	66H	65H
104H	04H	03H	02H	01H
	107H	106H	105H	104H

字地址	100H	101H	102H	103H
100H	65H	66H	67H	68H
104H	01H	02H	03H	04H
	104H	105H	106H	107H

存放ASCII码字符串和BCD码数

有差别
若是按小端方案存取就全反了
(大)
(小)

无差别
字地址一样,就会从左往右(从高位到低位)全部取出
按字节访问

边界对齐的数据存放方法

某微处理器能同时读入4个字节的数据,然而,这4个字节必须在连续的单元中,它们的地址除最低2位不同外,其余位均相同。

例如,该CPU可以同时读单元100H、101H、102H、103H,但不能同时读101H、102H、103H、104H。

需要2个操作,2次访问内存:

①读操作单元100H(无用)、101H、102H、103H

②读操作单元104H、105H(无用)、106H(无用)、107H(无用)

在采用字节编址的存储器中,数据有3种不同的存放方法,其中边界对齐的存放方法是最有效的方法。其对数据存放的位置有如下要求:

8位数据,占1个存储单元,起始地址为X...XXXX(任意)

16位数据,占2个存储单元,起始地址为X...XXX0(2的整数倍)

32位数据,占4个存储单元,起始地址为X...XX00(4的整数倍)

64位数据,占8个存储单元,起始地址为X...X000(8的整数倍)

存储器中的数据存放(存储字长为32位) 4个字节.

边界对准				地址(十进制)
字(地址0)				0
字(地址4)				4
字节(地址11)	字节(地址10)	字节(地址9)	字节(地址8)	8
字节(地址15)	字节(地址14)	字节(地址13)	字节(地址12)	12
半字(地址18)		半字(地址16)		16
半字(地址22)		半字(地址20)		20
双字) (地址24)				24
双字) (地址28)				28
双字) (地址32)				32
双字) (地址36)				36

边界未对准		地址(十进制)
低16位		
字节(地址7)	字节(地址6)	0
半字(地址10)		4
半字(地址8)		8

高16位 4 1个字要取2次,效率低!

例. 某机字长32位, 主存储器按字节编址, 现有4种不同长度的数据(字节、半字、单字、双字), 请采用一种既节省存储空间, 又能保证任何长度的数据都能在单个存储存取周期内完成读写的方法, 将一批数据顺序地存入主存, 画出主存中数据的存放示意图。

这批数据一共有10个, 依次为字节、半字、双字、单字、字节、单字、双字、半字、单字和字节。

解: 由题干可知4种长度的数据分别是:

字节数据8位, 半字数据16位, 单字数据32位, 双字数据64位,

由于要保证任何长度的数据都能在单个存储存取周期内完成读写,

故该机的存储器字长应是64位(与数据字长32位不同)

应采取边界对齐的存放方式。

字节地址(二进制) 大端方案

X...X000	X...X001	X...X010	X...X011	X...X100	X...X101	X...X110	X...X111
字节任意	半字 ...0						
双字 ...000							
单字 ...00				字节任意			
单字 ...00							
双字 ...000							
半字 ...0				单字 ...00			
字节任意							

操作类型

1. 数据传送

源	寄存器	寄存器	存储器	存储器
目的	寄存器	存储器	寄存器	存储器
例如	MOVE	STORE MOVE PUSH	LOAD MOVE POP	MOVE

助记符

置"1", 清"0"

2. 算术逻辑操作

加、减、乘、除、增1、减1、求补、浮点运算、十进制运算、与、或、非、异或、位操作、位测试、位清除、位求反

如8086 ADD SUB MUL DIV INC DEC CMP NEG
AAA AAS AAM AAD
AND OR NOT XOR TEST

3. 移位操作

算术移位 逻辑移位 循环移位 (带进位和不带进位)

4. 转移

① 无条件转移 JMP

② 条件转移

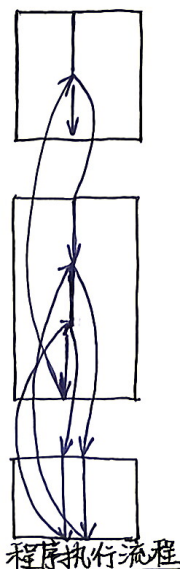
结果为零转 (Z=1) JZ
结果溢出转 (O=1) JO
结果有进位转 (C=1) JC
跳过一条指令 SKP

如 300
...
305 SKP DZ D=0 则跳
306
307

完成触发器

③ 调用和返回

地址 主程序
2000
2100 CALL SUB1
2101
子程序SUB1
2400
2500 CALL SUB2
2501
2560 CALL SUB2
2561 RETURN
子程序SUB2
2700
RETURN
主存空间分配



④ 陷阱(Trap)与陷阱指令

意外事故的中断.

- 一般不提供给用户直接使用
在出现事故时, 由CPU自动产生并执行(隐指令)
- 设置供用户使用的陷阱指令
如8086 INT TYPE 软中断
提供给用户使用的陷阱指令, 完成系统调用

5. 输入输出

入 端口地址 → CPU的寄存器
如 IN AX, n 从n号端口读取数据到AX寄存器
AL IN AX, DX 存放端口地址
出 CPU的寄存器 → 端口地址
如 OUT n, AX 将寄存器AX中的数据写入指定端口
OUT DX, AL

寻址方式

- 指令寻址 (比较简单)
- 数据寻址 (比较复杂)

确定本条指令的操作数地址

下一条欲执行指令的指令地址

在存储器中, 操作数或指令字写入或读出的方式, 有地址指定方式、相联存储方式、堆栈存取方式

几乎所有计算机, 在内存中都采用地址指定方式

当采用地址指定方式时, 形成操作数或指令地址的方式, 称为寻址方式.

指令寻址

用汇编语言/机器语言写的程序, 75%是顺序结构

顺序

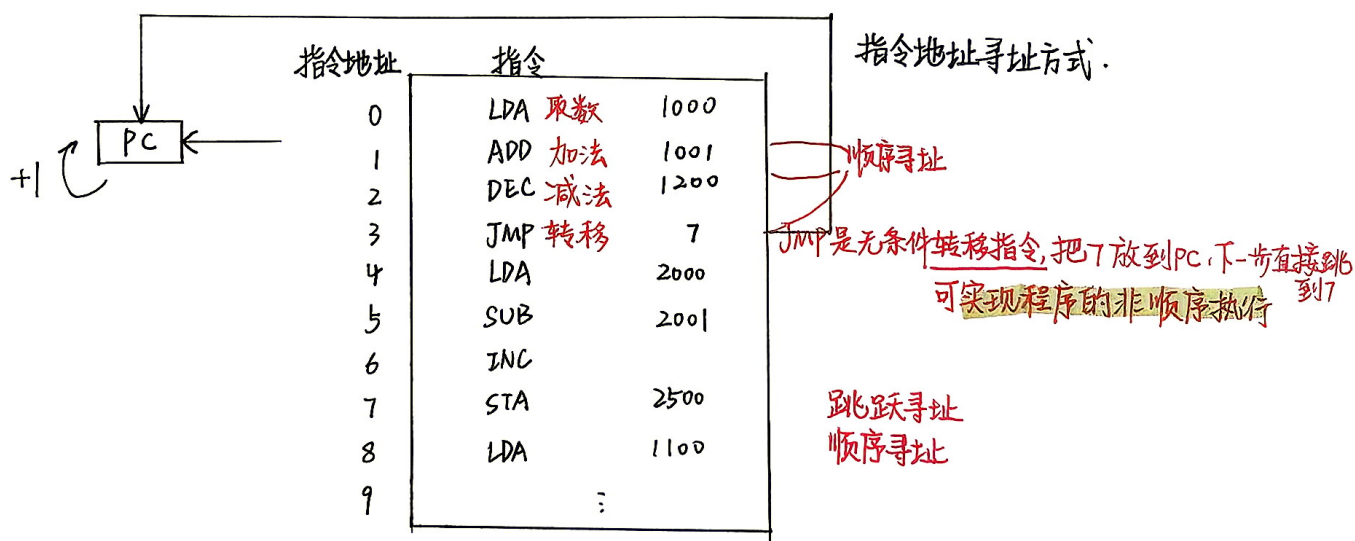
$(PC)+1 \rightarrow PC$ 专用寄存器, 用来存放指令的地址
相当于程序计数器

加“1”,
看这条指令占几个编址单位.

这里是单字长指令, 1个指令占1个编址单位

跳跃

由转移指令指出.



数据寻址

对寻址方式的编码

操作码	寻址特征	形式地址A
-----	------	-------

形式地址 指令字中的地址

有效地址 操作数的真实地址

注意! 取指令都有
一次访存

约定: 指令字长 = 存储字长 = 机器字长

立即寻址

形式地址A就是操作数

立即寻址特征

OP	#	A
----	---	---

A = 操作数

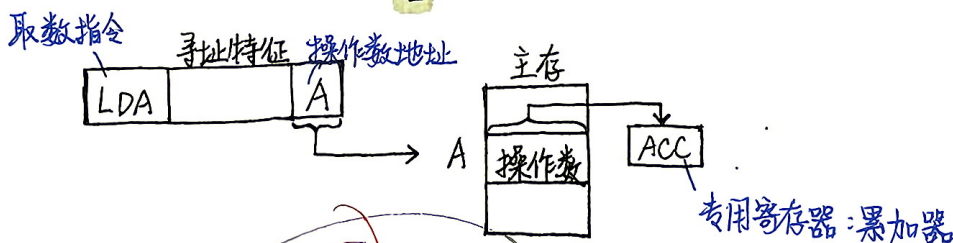
立即数(可正可负,补码)

优点: 执行速度快, 数据就在指令中, 指令执行阶段不访问内存

缺点: A的位数限制了立即数的范围, 比机器数表达的范围小很多

直接寻址

EA = A 有效地址由形式地址直接给出 EA = A



缺点: ① 执行阶段访问一次存储器

优点: 操作数范围大, 操作数长度就是存储器字长 = 机器字长, 表示范围与机器数表示范围一样

② A的位数决定了该指令操作数的寻址范围(假设A为6位, 那么其能表达的范围是0~63, 只有在主存中0~63的存储单元(最小的64个)才能找到)

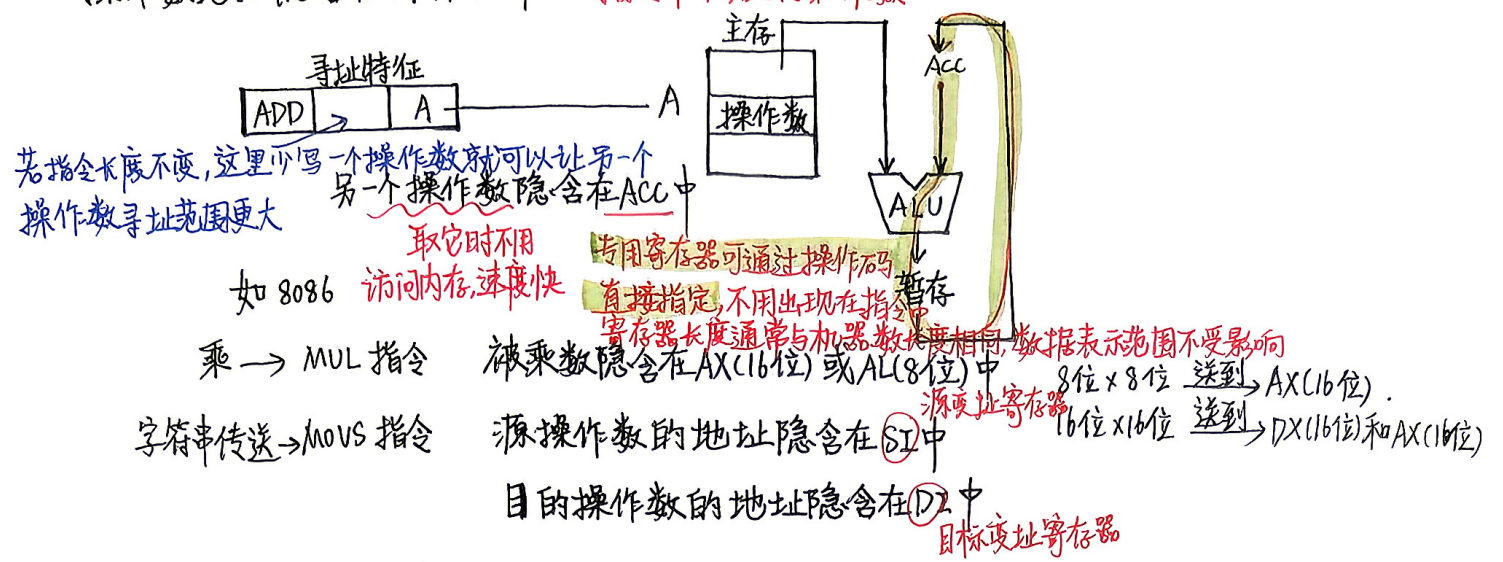
③ 操作数的地址不易修改(必须修改A)

就要修改指令, 相当于重新修改程序

隐含寻址 (不用访问内存, 速度快).

操作数在专用寄存器

操作数地址隐含在操作码中 指令中不给出操作数



乘 → MUL 指令 被乘数隐含在AX(16位)或AL(8位)中
 字符串传送 → MOVSB 指令 源操作数的地址隐含在SI中
 目的操作数的地址隐含在DI中

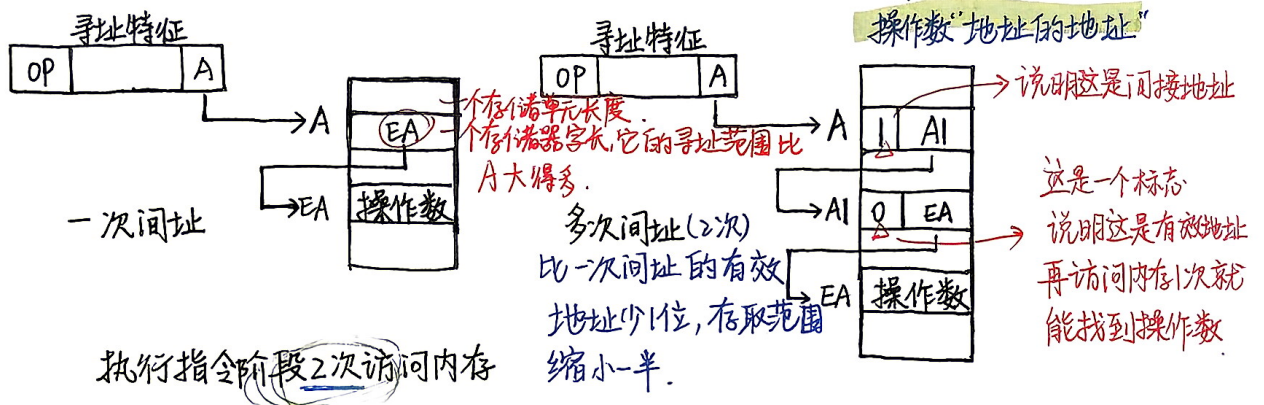
指令字中少了一个地址字段, 可缩短指令字长

间接寻址

EA = (A)

有效地址由形式地址间接提供

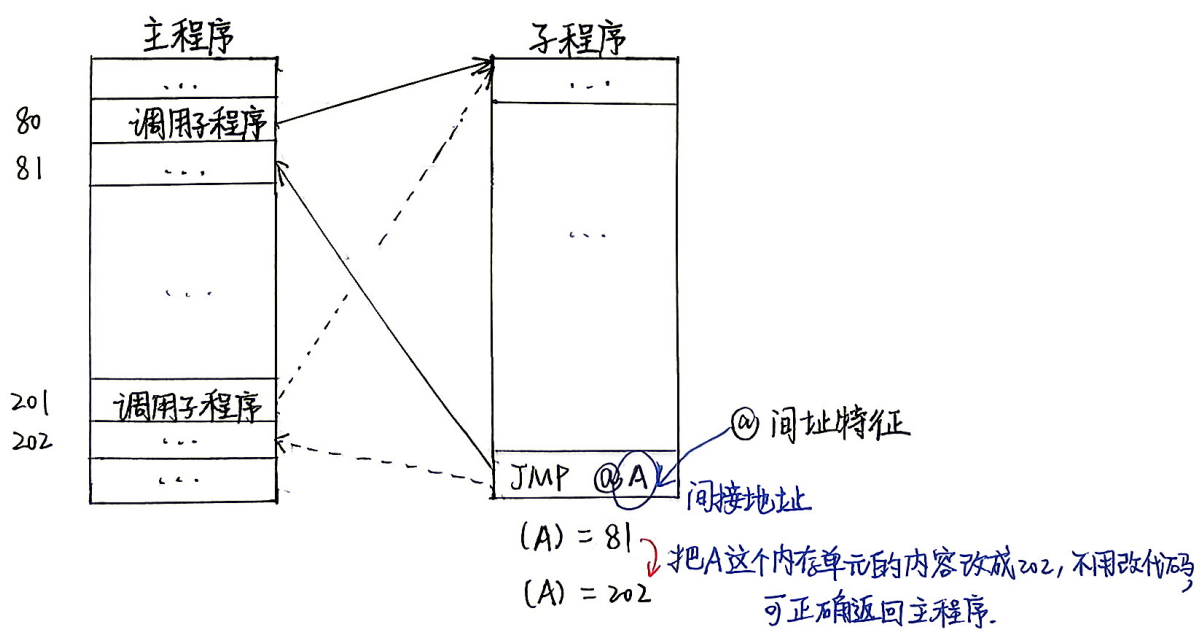
指令中的地址是操作数“地址的地址”



执行指令阶段2次访问内存
 可以扩大寻址范围
 便于编制程序

多次访存
 3次

编程举例



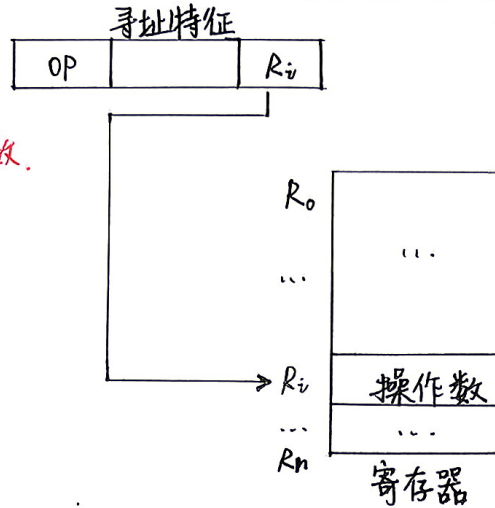
寄存器寻址

$$EA = R_i$$

长度=4位 若有16个,要4位并行编码
有效地址是寄存器编号

通用寄存器,同种类型的有很多.

寄存器中放操作数.



执行阶段不访问内存,只访问寄存器,执行速度快(认为其速度就是运算器速度)

寄存器个数有限,可缩短指令字长

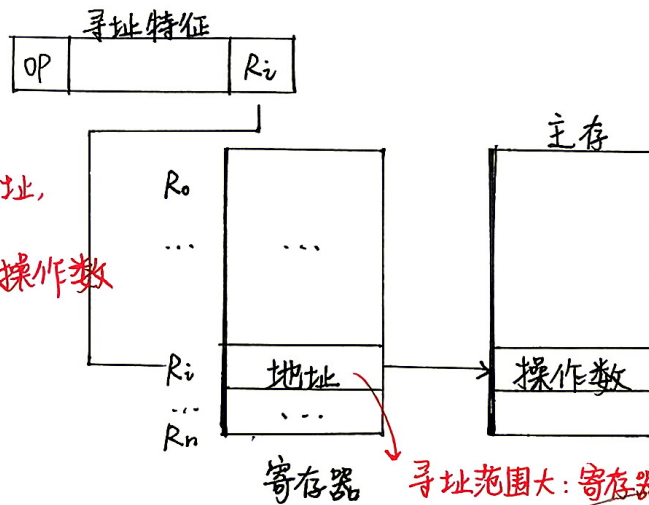
寄存器间接寻址

$$EA = (R_i)$$

有效地址在寄存器中

寄存器中放操作数地址,

再到主存中找操作数



寻址范围: 寄存器长度 = 指令字长 = 机器字长

有效地址在寄存器中,操作数在存储器中,执行阶段访问内存

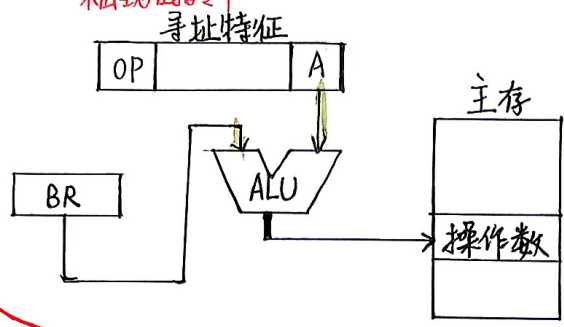
便于编制循环程序

基址寻址 (有效地址由基址寄存器和形式地址构成)

记下每一条指令被装入内存的首地址

① 采用专用寄存器作基址寄存器

不变 / 偏移量, 可变
 $EA = (BR) + A$
 BR 为基址寄存器
 不出现在指令中



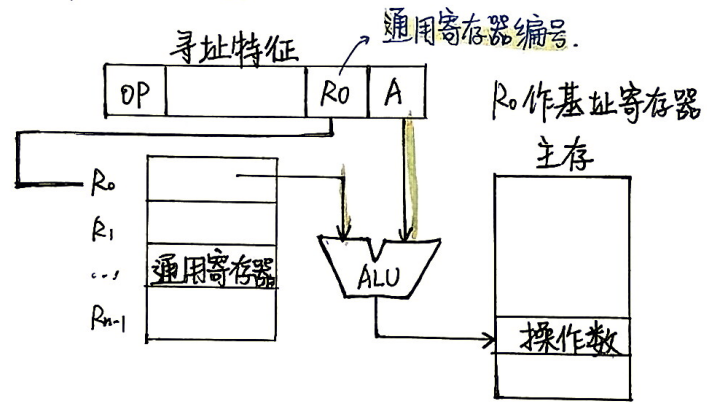
可扩大寻址范围

有利于多道程序并行

BR 内容由操作系统或管理程序确定

在程序的执行过程中 BR 内容不变, 形式地址 A 可变

② 采用通用寄存器作基址寄存器



由用户指定哪个通用寄存器作基址寄存器

基址寄存器的内容由操作系统确定

在程序的执行过程中 R0 内容不变, 形式地址 A 可变

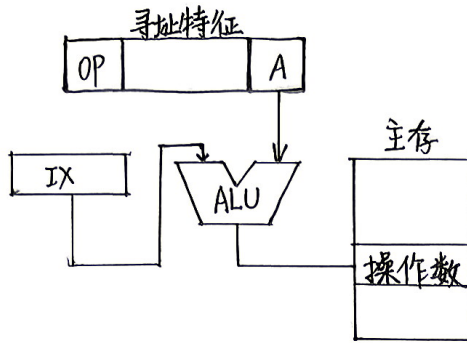
通过修改形式地址来获得
不同位置的指令或数据

变址寻址

EA = (IX) + A

IX 为变址寄存器(专用)

通用寄存器也可以作为变址寄存器



可扩大寻址范围 由 IX 和 A 两部分加起来形成, 比 A 大

IX 的内容由用户给定 第1个元素 IX 是 0. 通过修改 IX 获得不同位置的元素.

在程序执行过程中 IX 内容可变, 形式地址 A 不变

便于处理数组问题 (数组名即形式地址, 数组元素的下标即变址寄存器的内容)
数值型 ✓ 字符串 ✓
首地址

例. 设数据块首地址为 D, 求 N 个数的平均值

直接寻址

把第1个数放入 ACC LDA D D(0) → ACC

隐含寻址. 第1个, 第2个加法结果放入 ACC ADD D+1 ACC + D(1) → ACC

ADD D+2 ACC + D(2) → ACC

⋮

...

ACC + D(n-1) → ACC

N 个数的和都在 ACC 里 ADD D+(N-1) 立即数 (立即寻址方式中指令直接给出的数值)

DIV 除法 #N ACC / N → ACC 隐含寻址. ACC 是被除数, 除完结果放入 ACC

STA ANS ACC → ANS 存数指令, 把 ACC 内容放入内存. ANS 是存放结果的地址的代号

共 (N+2) 条指令

better:

变址寻址

LDA #0

0 → ACC

取数, 把 0 这个立即数送入 ACC, 让 ACC 清零

LDX #0

0 → X

把 0 送入变址寄存器 X

ADD X, D

ACC + (D + X) → ACC 把 ACC + [形式地址 D + X] 送回 ACC

执行 N 次

INX

X + 1 → X

内存单元地址

CPX #N

将变址寄存器中 X 与立即数 #N 比较看是否相等.

BNE M

BNE: 不相等 (不为 0) 就跳转

DIV #N

ACC / N → ACC

STA ANS

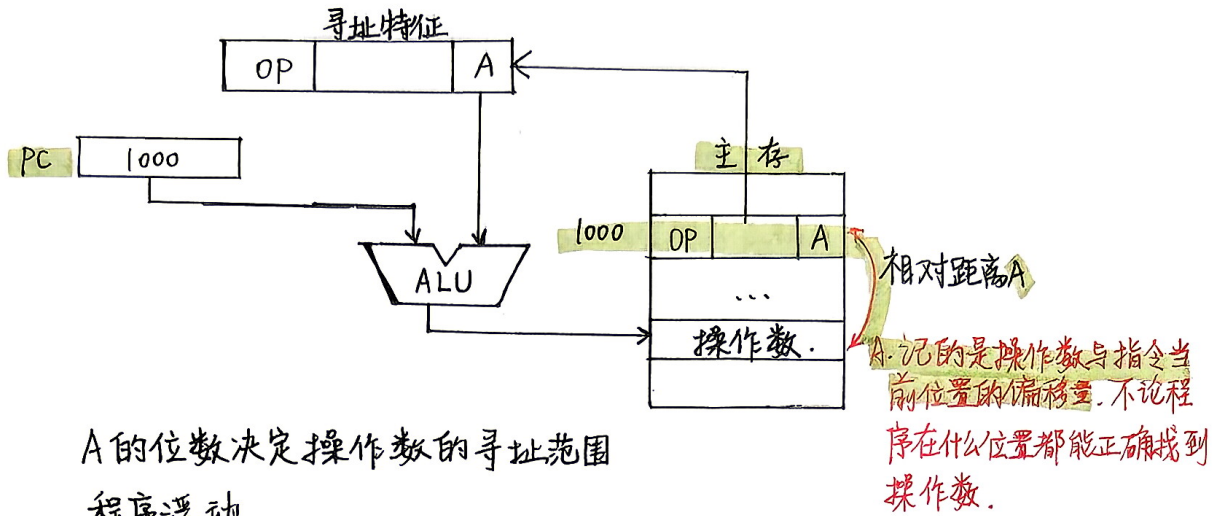
ACC → ANS

共 8 条指令

相对寻址 → 有效地址 ← 程序计数器. 当前指令的地址

$EA = (PC) + A$ → 指令中的形式地址

A 是相对于当前指令的位移量 (可正可负, 补码)



① 相对寻址举例

LDA #0
LDX #0
ADD X, D
INX
CPX #N
BNE (M) → * - 3
DIV #N
STA ANS

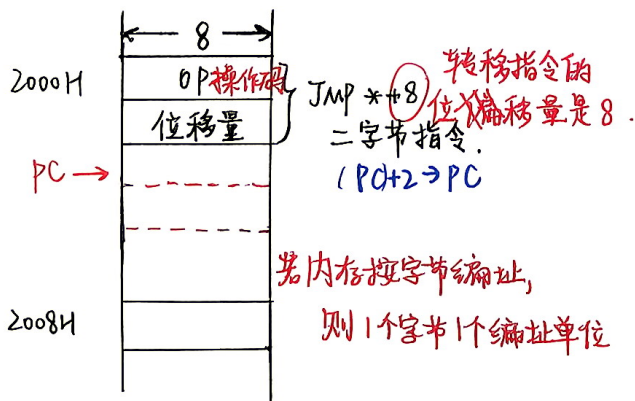
→ M
M+3

相对寻址特征
在 PC 中.

M 随程序所在存储空间的位置不同而不同, 而指令 BNE * - 3 与指令 ADD X, D 的相对位移量不变.

指令 BNE * - 3 操作数的有效地址是 $EA = (M+3) - 3 = M$

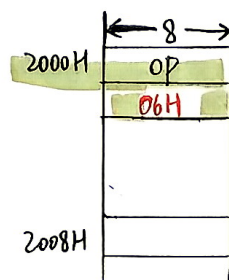
② 按字节寻址的相对寻址举例



取出 JMP *+8 后, PC = 2002H.

故 JMP *+8 指令的第 2 字节为

$2008H - 2002H = 06H$



设当前指令地址 PC = 2000H

转移后的目的地址为 2008H

以上 3 种十分相似: 有效地址 = 寄存器的值 + 指令中给出的形式地址

堆栈寻址

硬堆栈 多个寄存器
软堆栈 指定的存储空间

SP的修改与主存编址方法有关。

访问一次内存就是访问一个存储单元

按字编址就是±1。

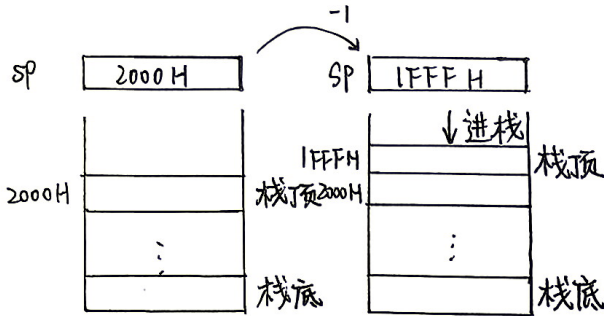
先进后出 (一个入出口)

栈顶地址由SP指出

栈顶指针

以“上小下大”为例:

进栈 $(SP) - 1 \rightarrow SP$



① 按字编址

进栈 $(SP) - 1 \rightarrow SP$

出栈 $(SP) + 1 \rightarrow SP$

② 按字节编址

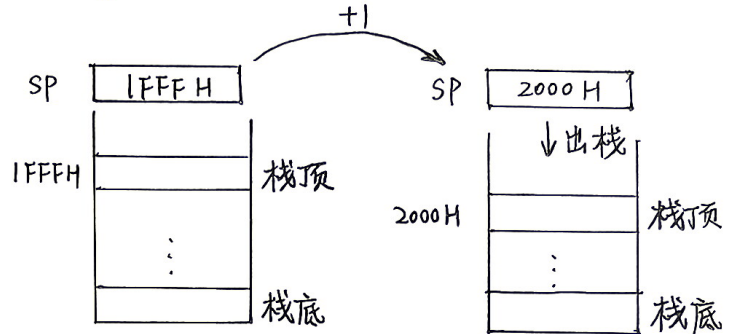
存储字长16位

$(SP) - 2 \rightarrow SP ; (SP) + 2 \rightarrow SP$

4个字节一个编址单位 → 存储字长32位

$(SP) - 4 \rightarrow SP ; (SP) + 4 \rightarrow SP$

出栈 $(SP) + 1 \rightarrow SP$



相当于寄存器间接寻址。

进栈: 先改指针, 再放数据

出栈: 先取数据, 再改指针

例.

